

# CSE 4621

# Machine Learning

**Topic: Dimensionality Reduction (PCA)**

Source: [DeepLearning.AI](https://deeplearning.ai)

**Presented by**

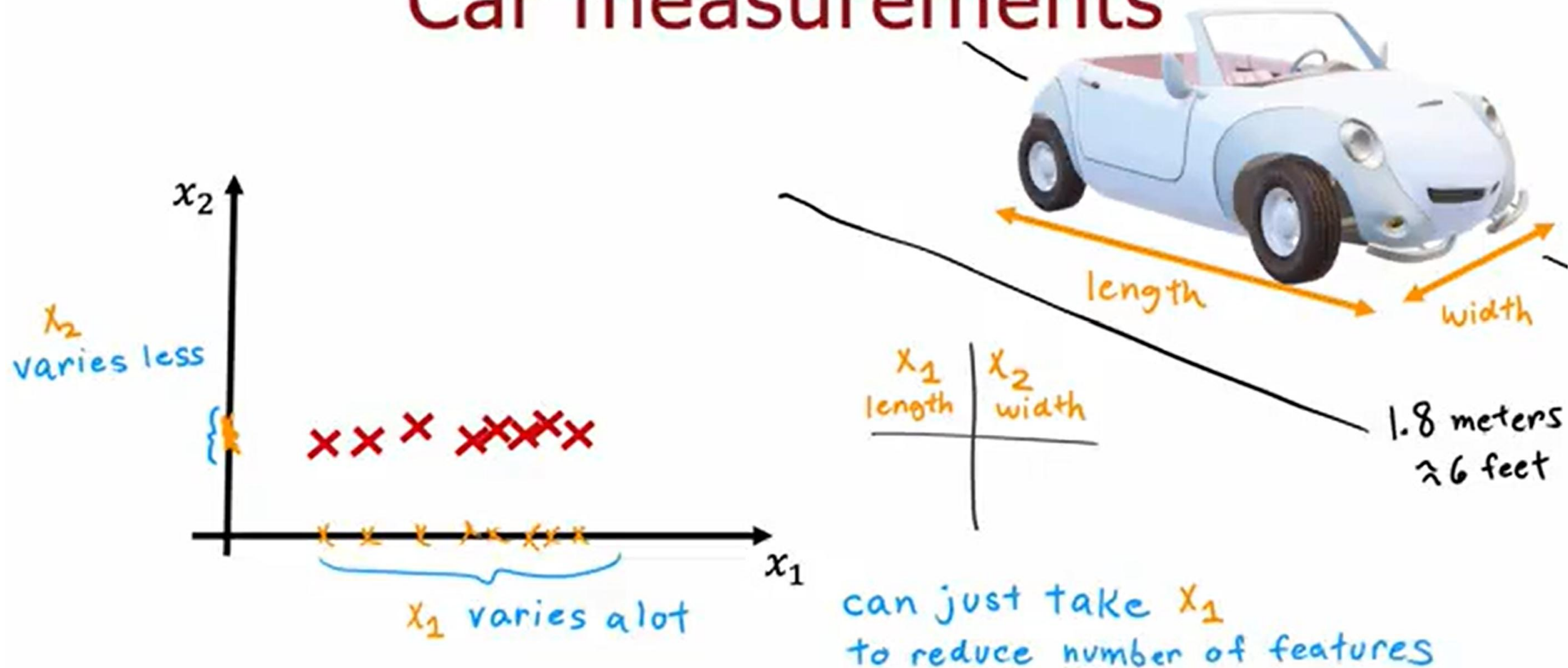
**Rafsanjany Kushol**

Assistant Professor



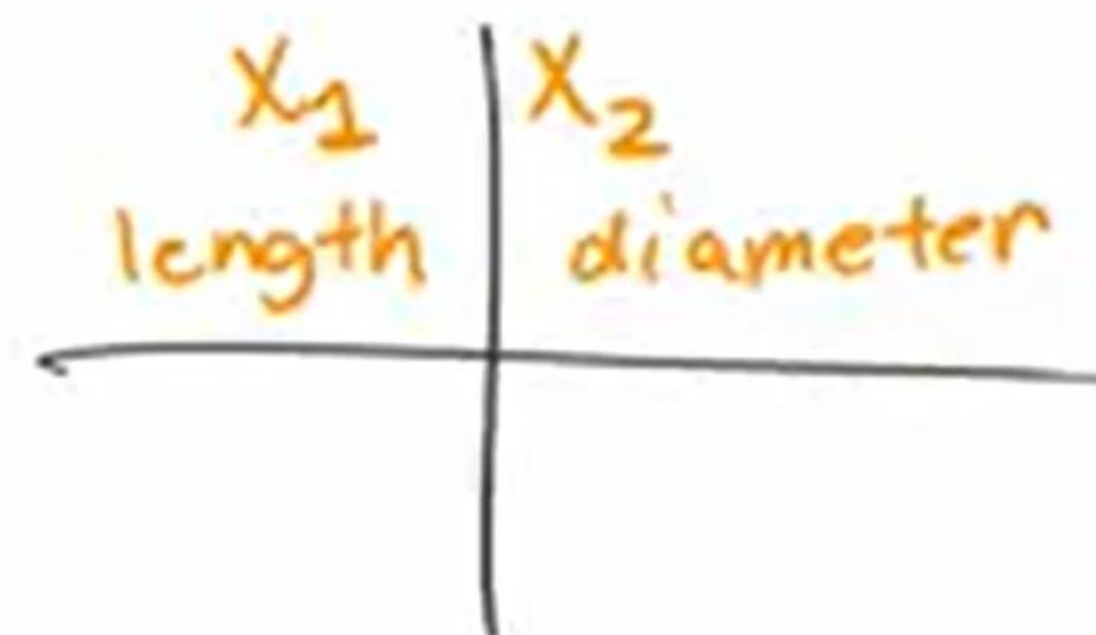
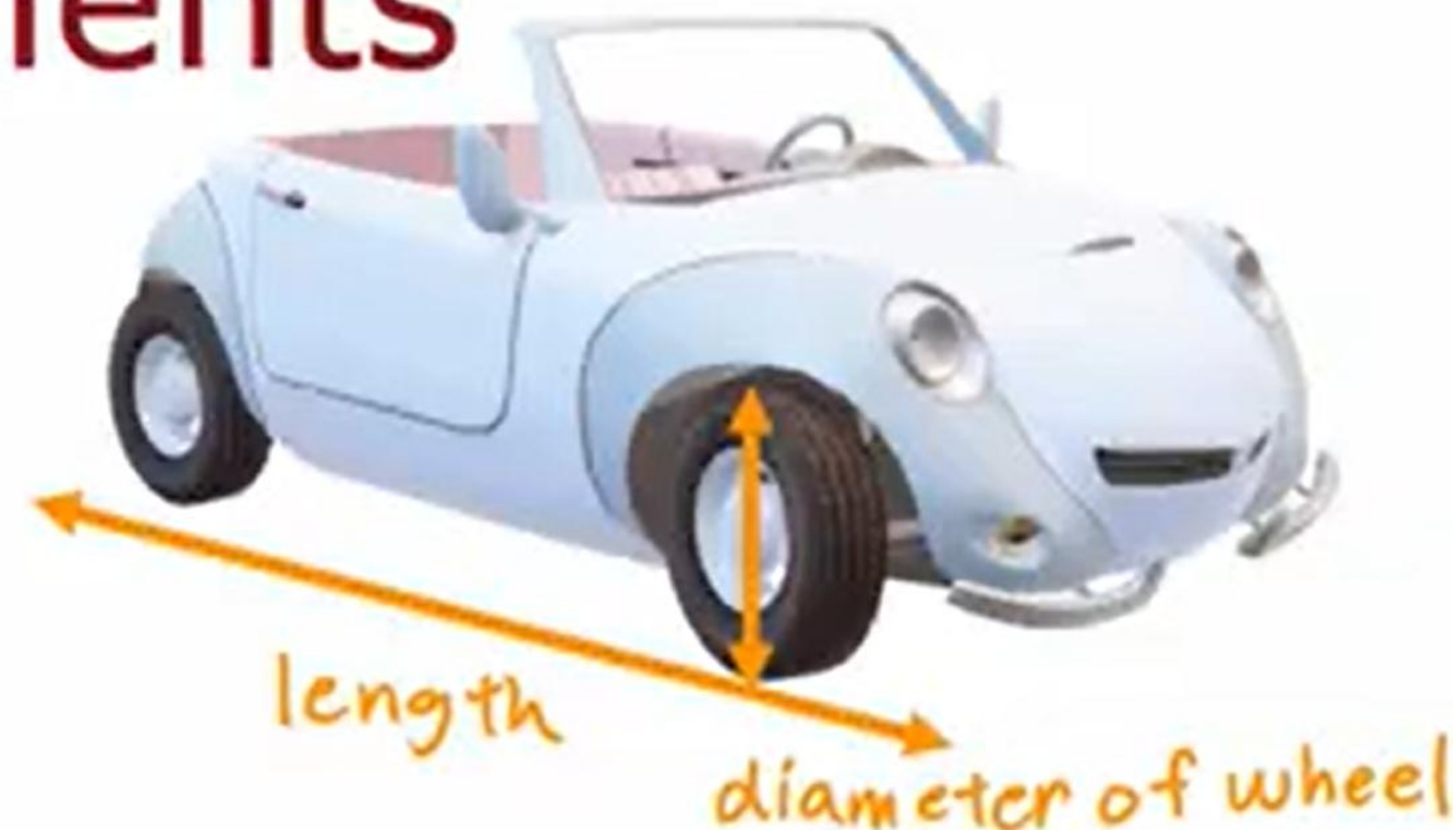
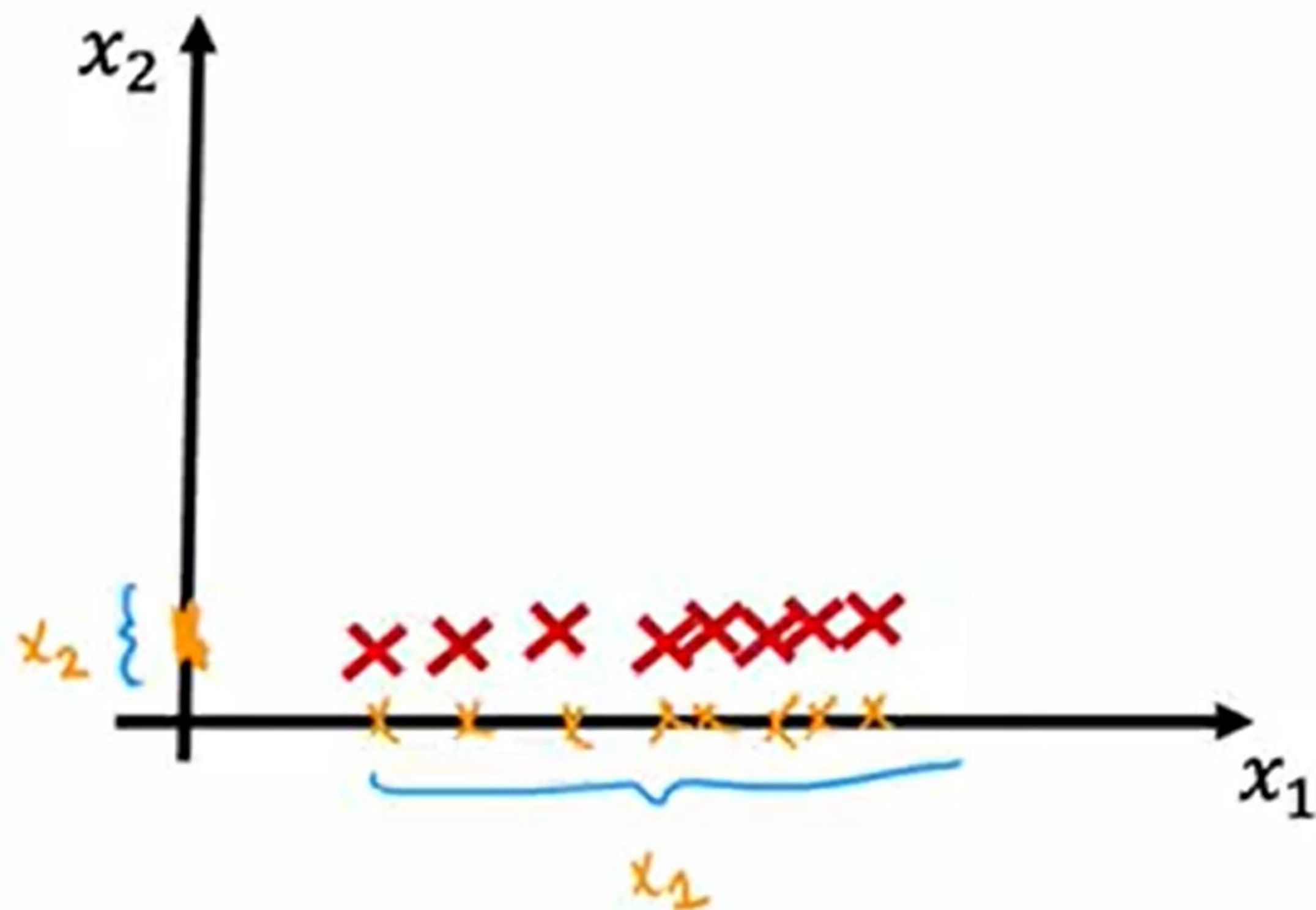
# Dimensionality Reduction

## Car measurements



# Dimensionality Reduction

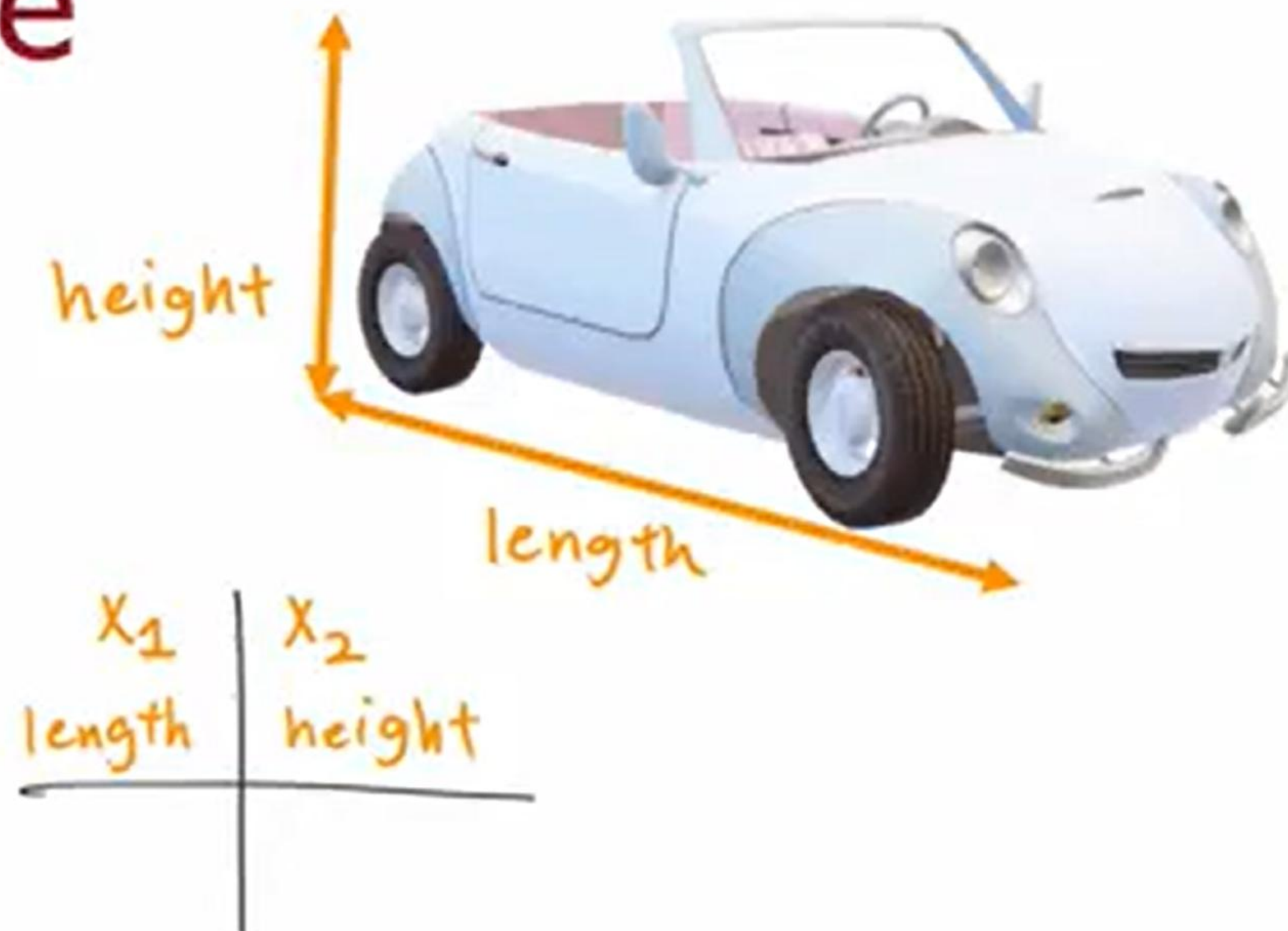
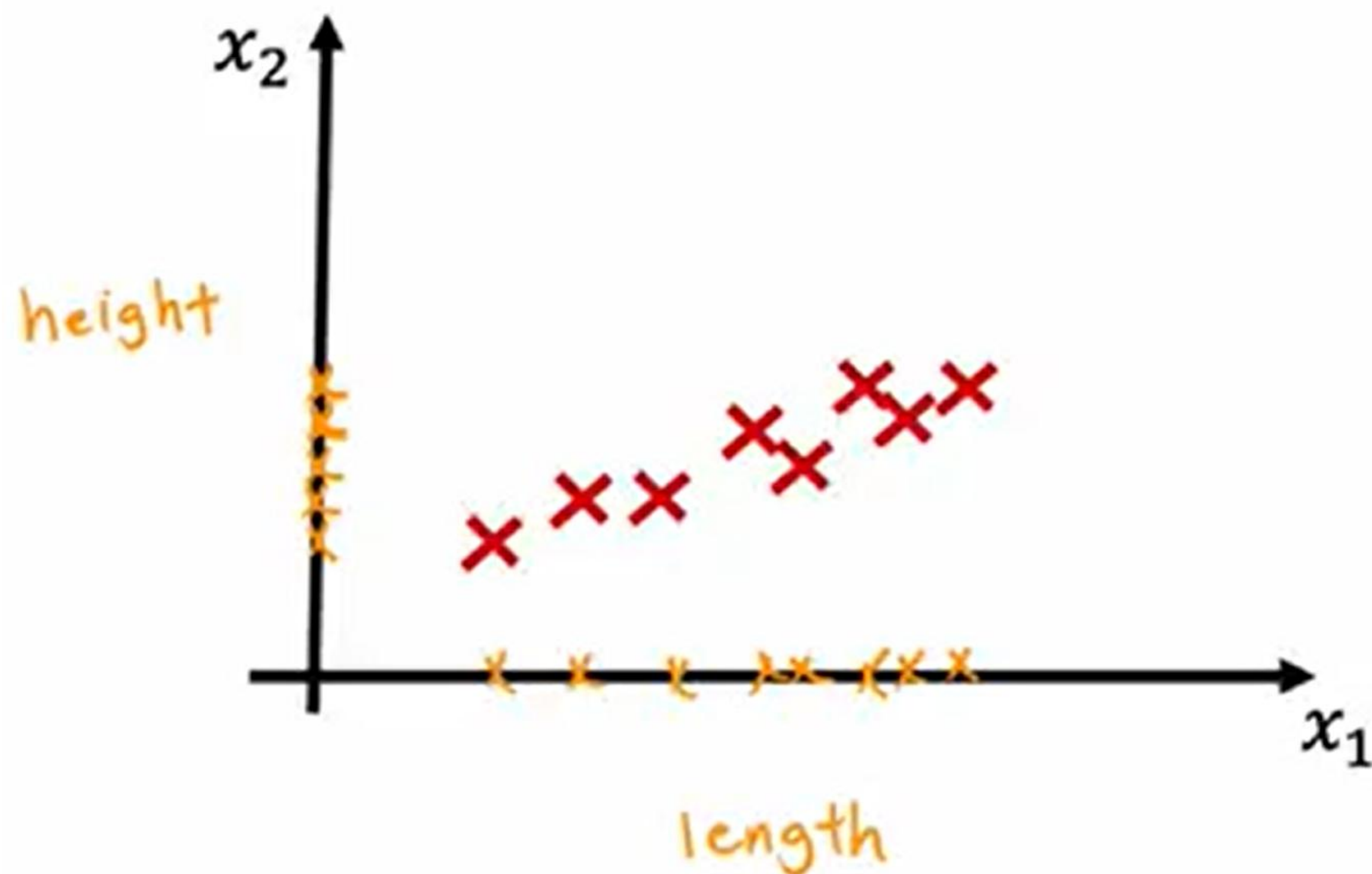
## Car measurements



can just take  $x_1$   
to reduce number of features

# Dimensionality Reduction

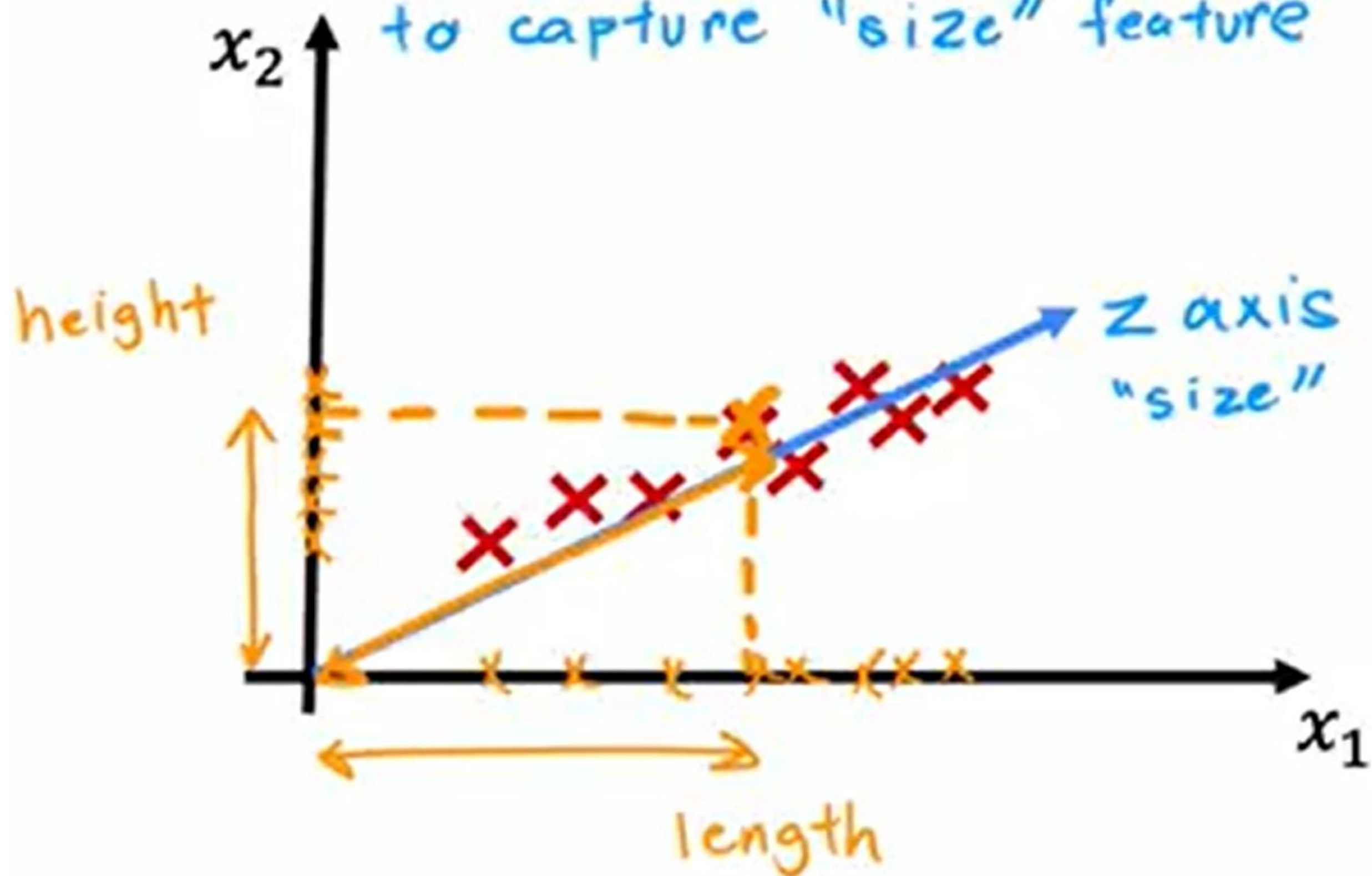
Size



# Dimensionality Reduction

## Size

PCA: find new axis and coordinates  
use fewer numbers  
to capture "size" feature

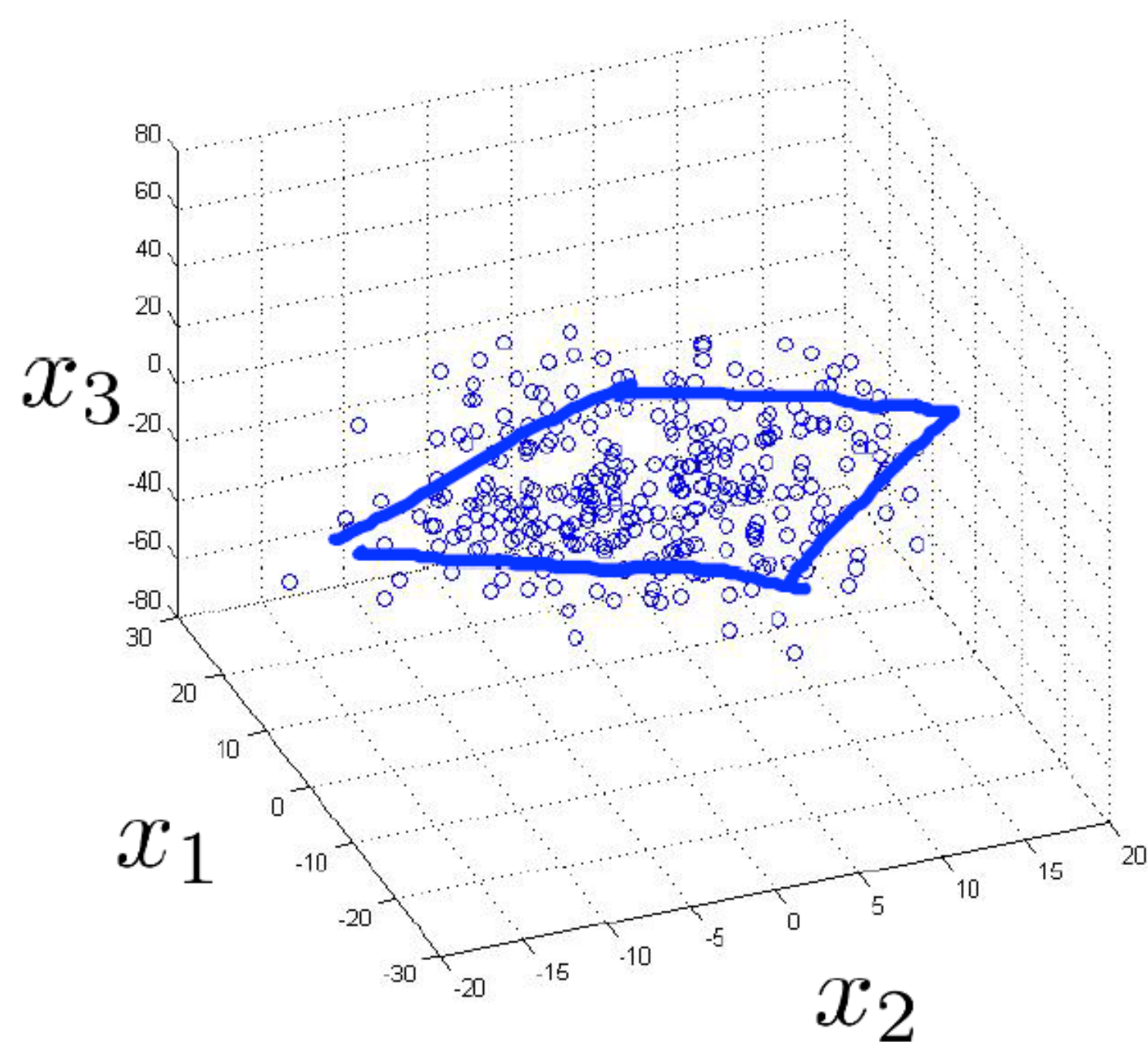


2 → 1 features

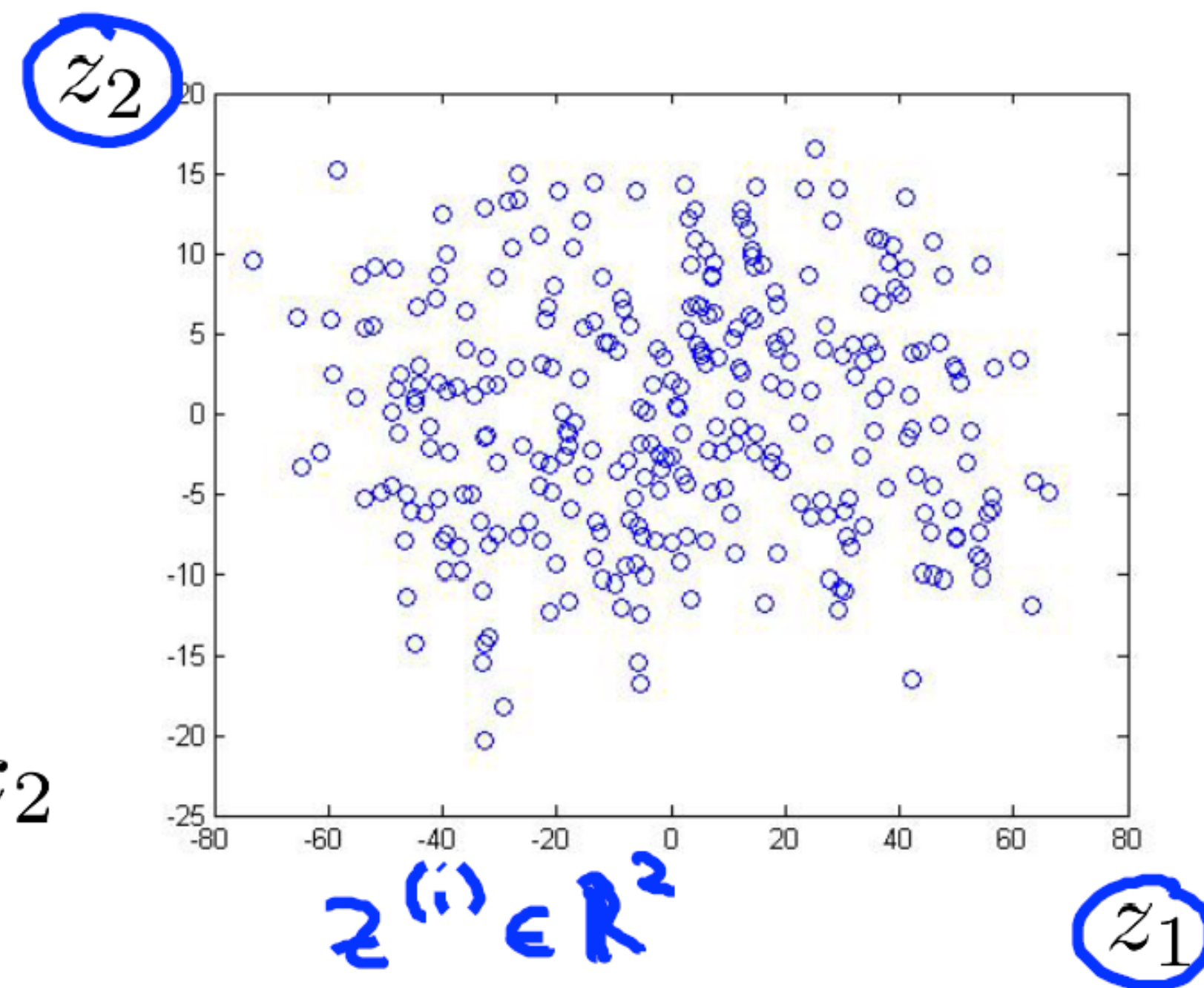
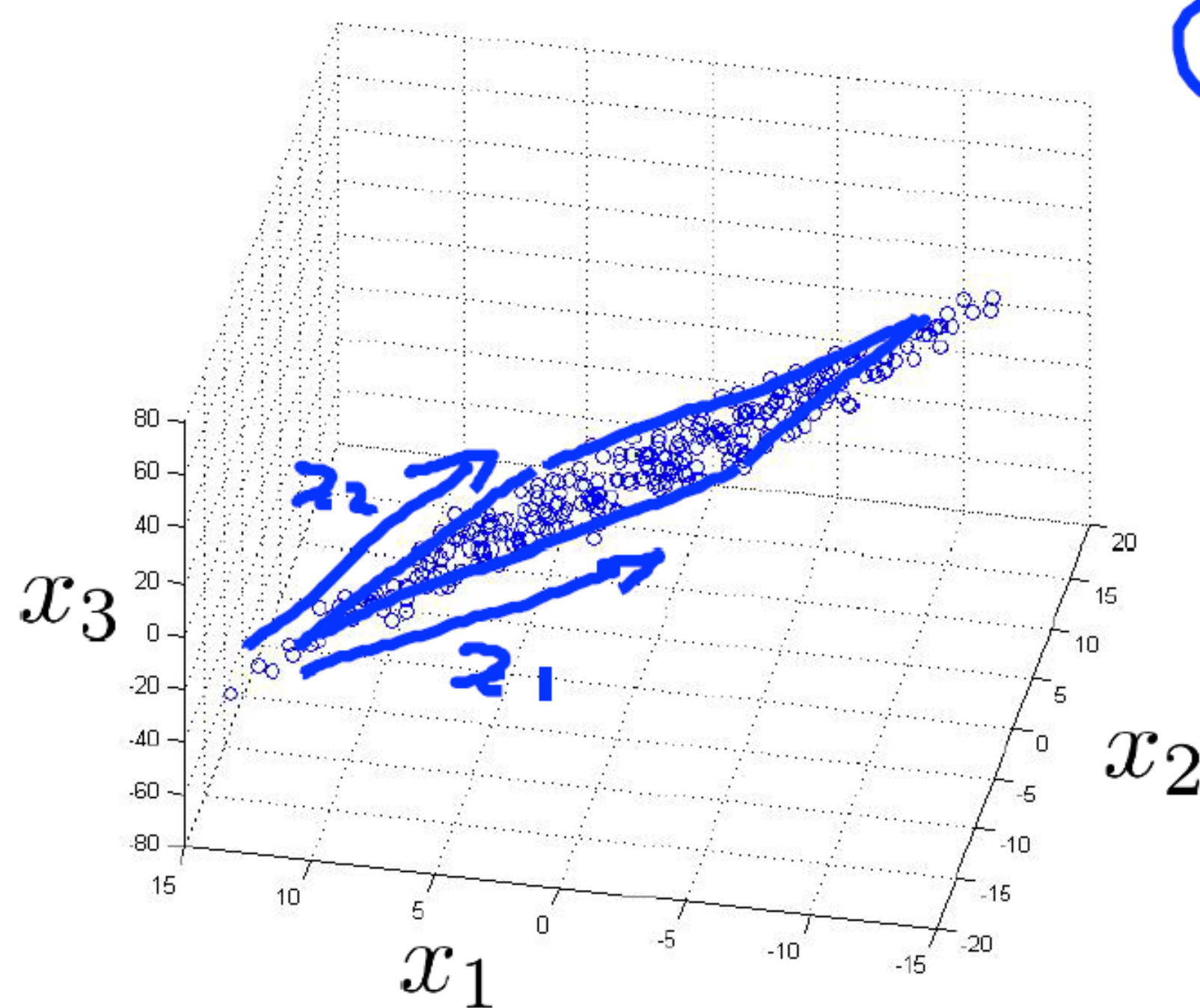
many features → 2 or 3 features

# Dimensionality Reduction

Reduce data from 3D to 2D



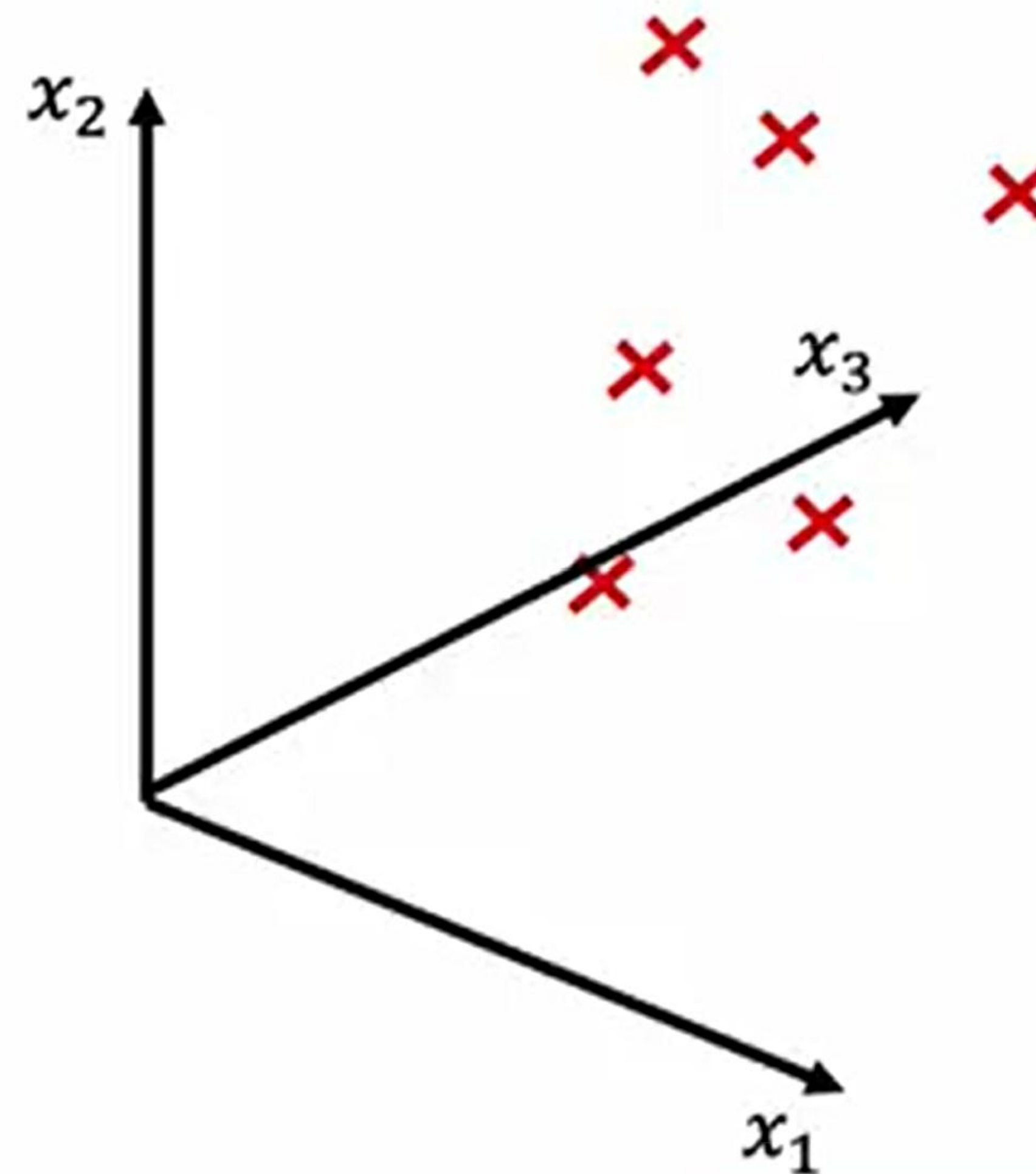
$$x^{(i)} \in \mathbb{R}^3$$



$$z = \begin{bmatrix} z_1 \\ z_2 \end{bmatrix} \quad z^{(i)} = \begin{bmatrix} z_1^{(i)} \\ z_2^{(i)} \end{bmatrix}$$

# Dimensionality Reduction

| Country   | GDP (trillions of US\$) | Per capita GDP (thousands of intl. \$) | Human Development Index |
|-----------|-------------------------|----------------------------------------|-------------------------|
| Canada    | 1.577                   | 39.17                                  | 0.908                   |
| China     | 5.878                   | 7.54                                   | 0.687                   |
| India     | 1.632                   | 3.41                                   | 0.547                   |
| Russia    | 1.48                    | 19.84                                  | 0.755                   |
| Singapore | 0.223                   | 56.69                                  | 0.866                   |
| USA       | 14.527                  | 46.86                                  | 0.91                    |
| ...       | ...                     | ...                                    | ...                     |



# Dimensionality Reduction

| Country   | GDP (trillions of US\$) | Per capita GDP (thousands of intl. \$) | Human Development Index | Life expectancy |
|-----------|-------------------------|----------------------------------------|-------------------------|-----------------|
| Canada    | 1.577                   | 39.17                                  | 0.908                   | 80.7            |
| China     | 5.878                   | 7.54                                   | 0.687                   | 73              |
| India     | 1.632                   | 3.41                                   | 0.547                   | 64.7            |
| Russia    | 1.48                    | 19.84                                  | 0.755                   | 65.5            |
| Singapore | 0.223                   | 56.69                                  | 0.866                   | 80              |
| USA       | 14.527                  | 46.86                                  | 0.91                    | 78.3            |
| ...       | ...                     | ...                                    | ...                     | ...             |

what if 50 features?

# Dimensionality Reduction

50 features

| Country | $X_1$<br>GDP (trillions of US\$) | $X_2$<br>Per capita GDP (thousands of Intl. \$) | $X_3$<br>Human Development Index | $X_4$<br>Life expectancy | ... | Poverty Index (Gini as percentage) | Mean household income (thousands of US\$) |
|---------|----------------------------------|-------------------------------------------------|----------------------------------|--------------------------|-----|------------------------------------|-------------------------------------------|
| Canada  | 1.577                            | 29.17                                           | 0.908                            | 80.7                     |     | 32.6                               | 67.293                                    |
| China   | 5.878                            | 7.54                                            | 0.687                            | 73                       |     | 46.9                               | 10.22                                     |
| India   | 1.632                            | 3.41                                            | 0.547                            | 64.7                     |     | 36.8                               | 0.735                                     |
| Russia  | 1.48                             | 19.84                                           | 0.755                            | 65.5                     |     | 39.9                               | 9.72                                      |

| Country | $Z_1$ | $Z_2$ |
|---------|-------|-------|
| Canada  | 1.6   | 1.2   |
| China   | 1.7   | 0.3   |
| India   | 1.6   | 0.2   |
| Russia  | 1.4   | 0.5   |



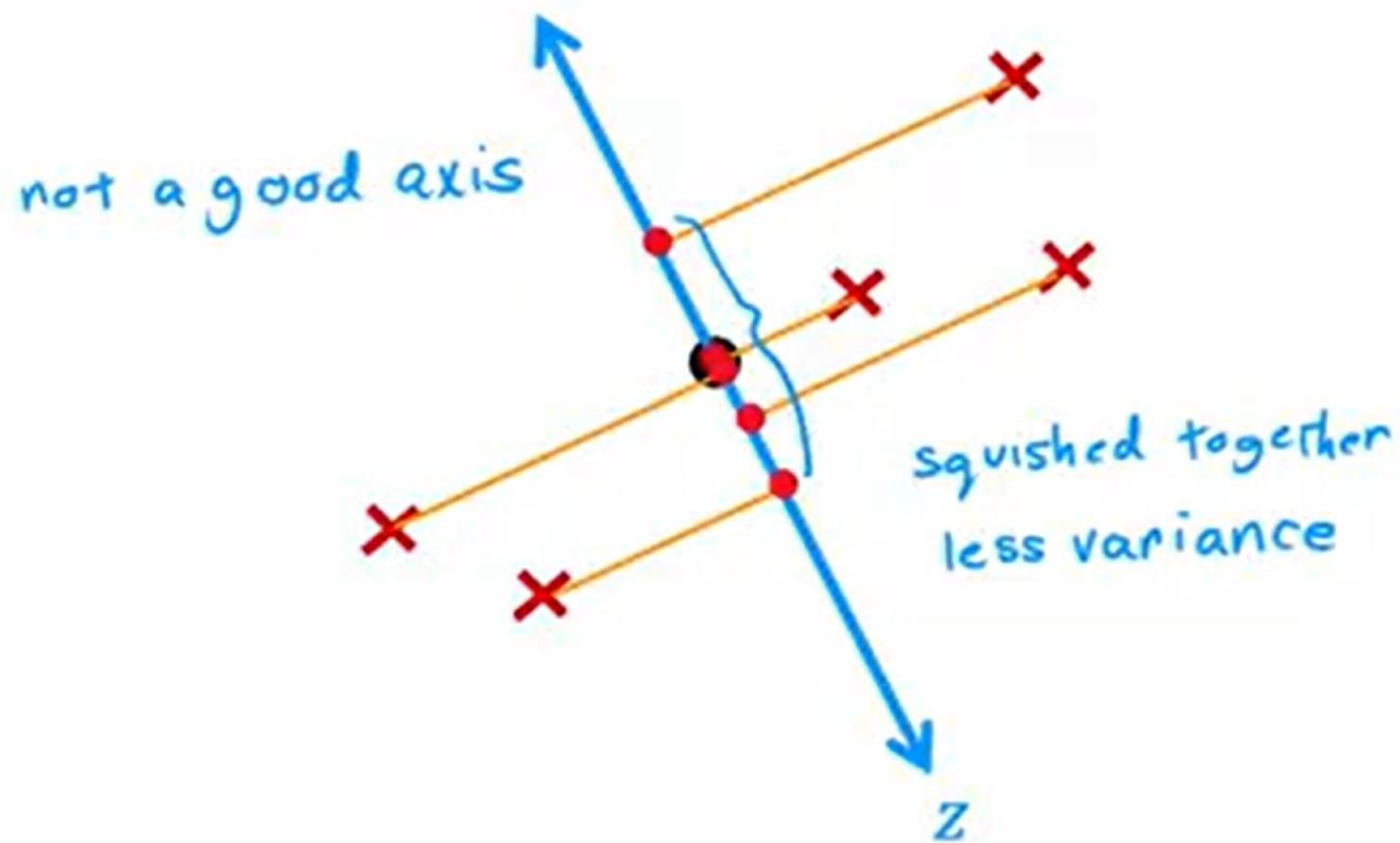
# Choose an axis

"project"  
examples  
onto the axis

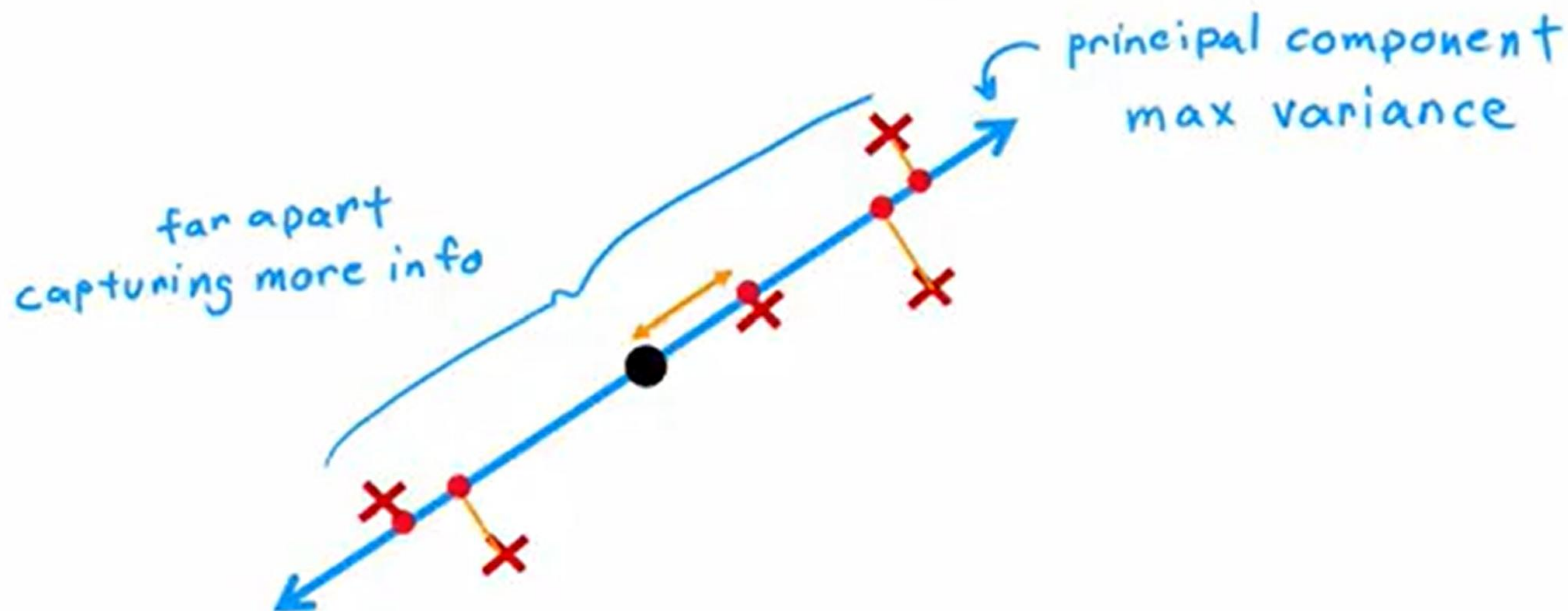


variance is large  
capturing info  
of original data

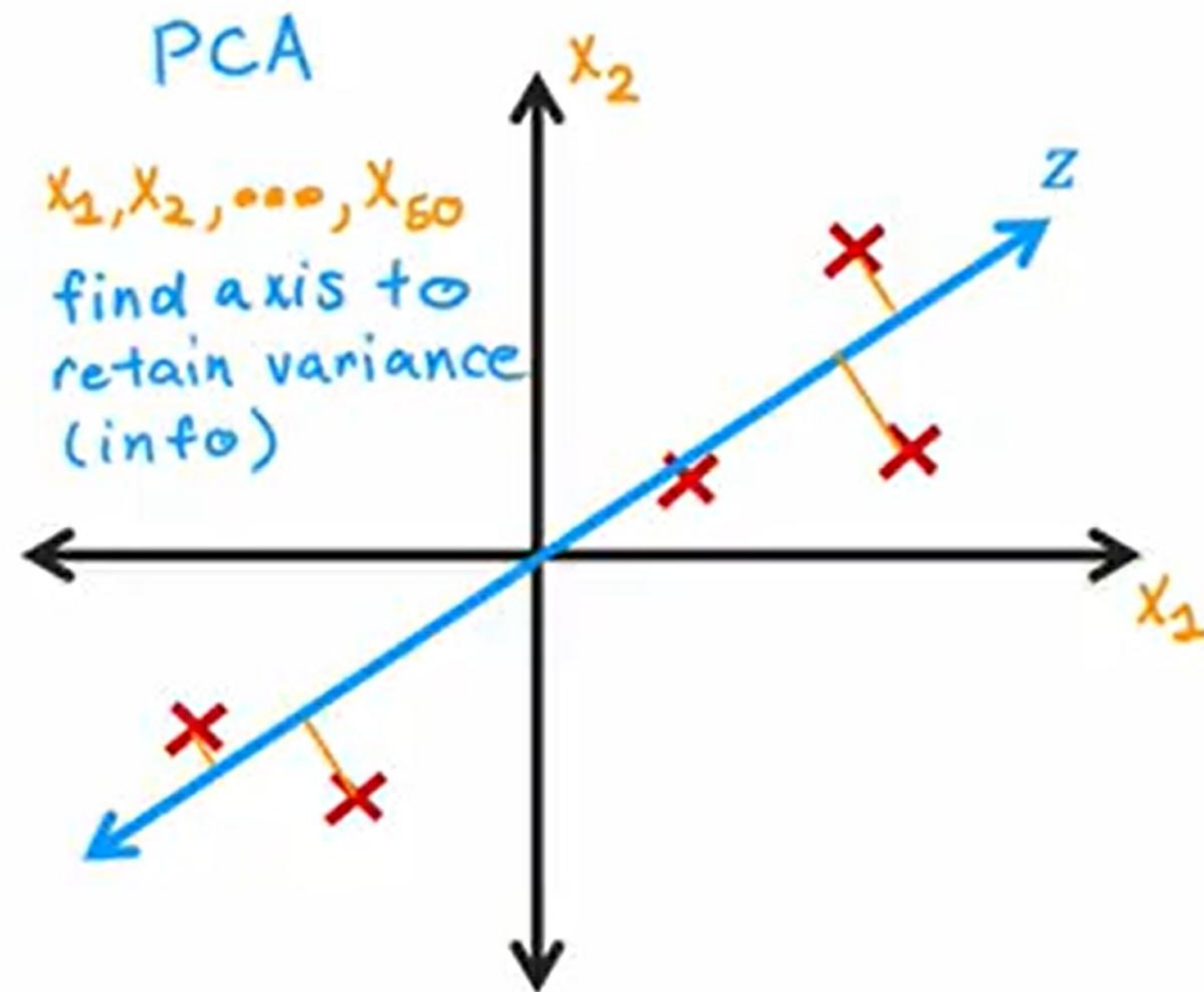
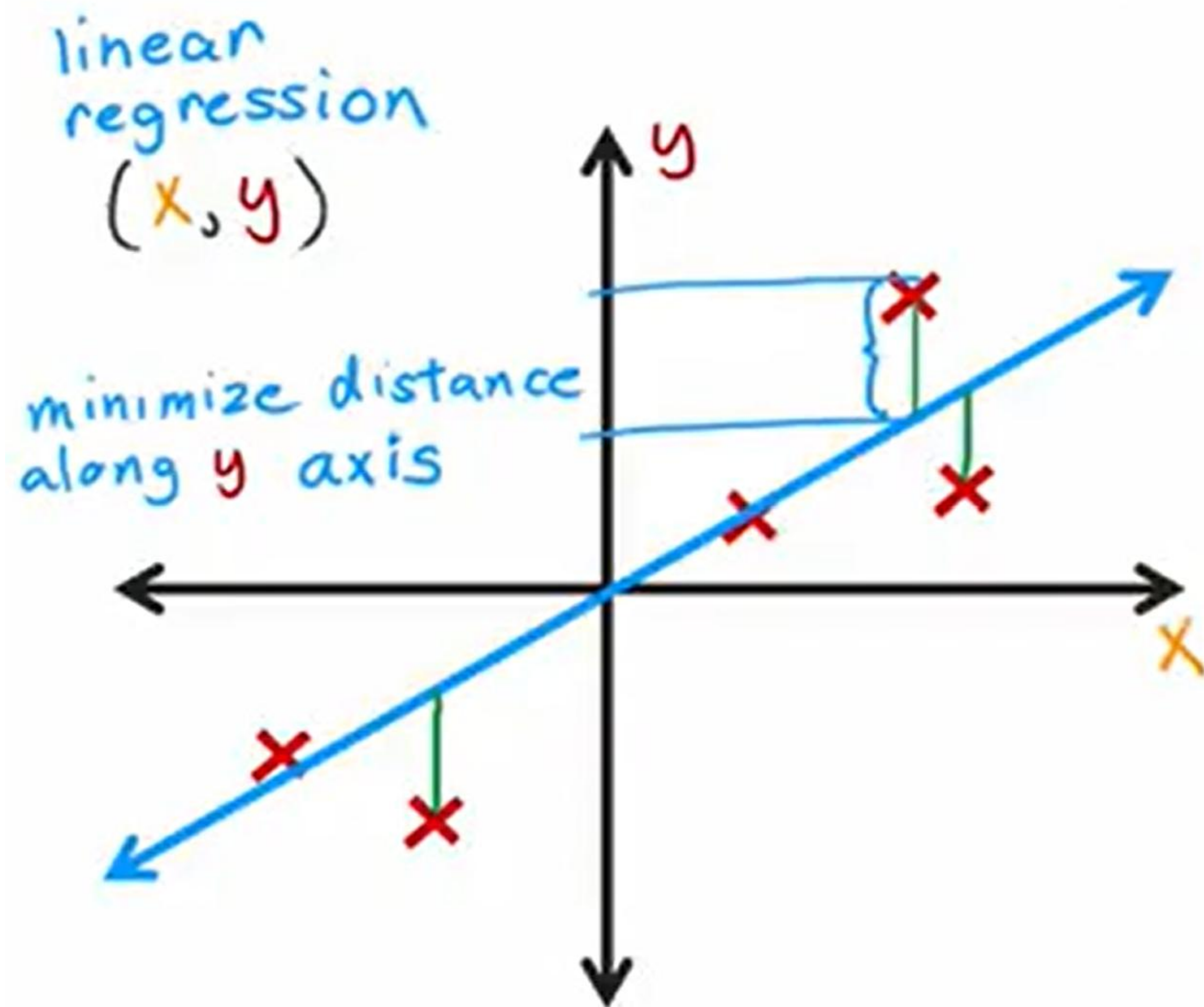
# Choose an axis



# Choose an axis



# PCA is not linear regression



# **Principal Component Analysis algorithm**

## **Steps**

- 1. Subtract the mean**
- 2. Calculate the covariance matrix**
- 3. Calculate the eigenvectors and eigenvalues**
- 4. Choosing components and forming a feature vector**

# Principal Component Analysis algorithm

## 1. Subtract Mean

Training set:  $x^{(1)}, x^{(2)}, \dots, x^{(m)}$

Preprocessing (feature scaling/mean normalization):

$$\mu_j = \frac{1}{m} \sum_{i=1}^m x_j^{(i)}$$

- Replace each  $x_j$  with  $(x_j - \mu_j)$ .
- If different features on different scales (e.g.,  $x_1$ =size of house,  $x_2$ =number of bedrooms), scale features to have comparable range of values.

# Principal Component Analysis algorithm

## 1. Subtract Mean

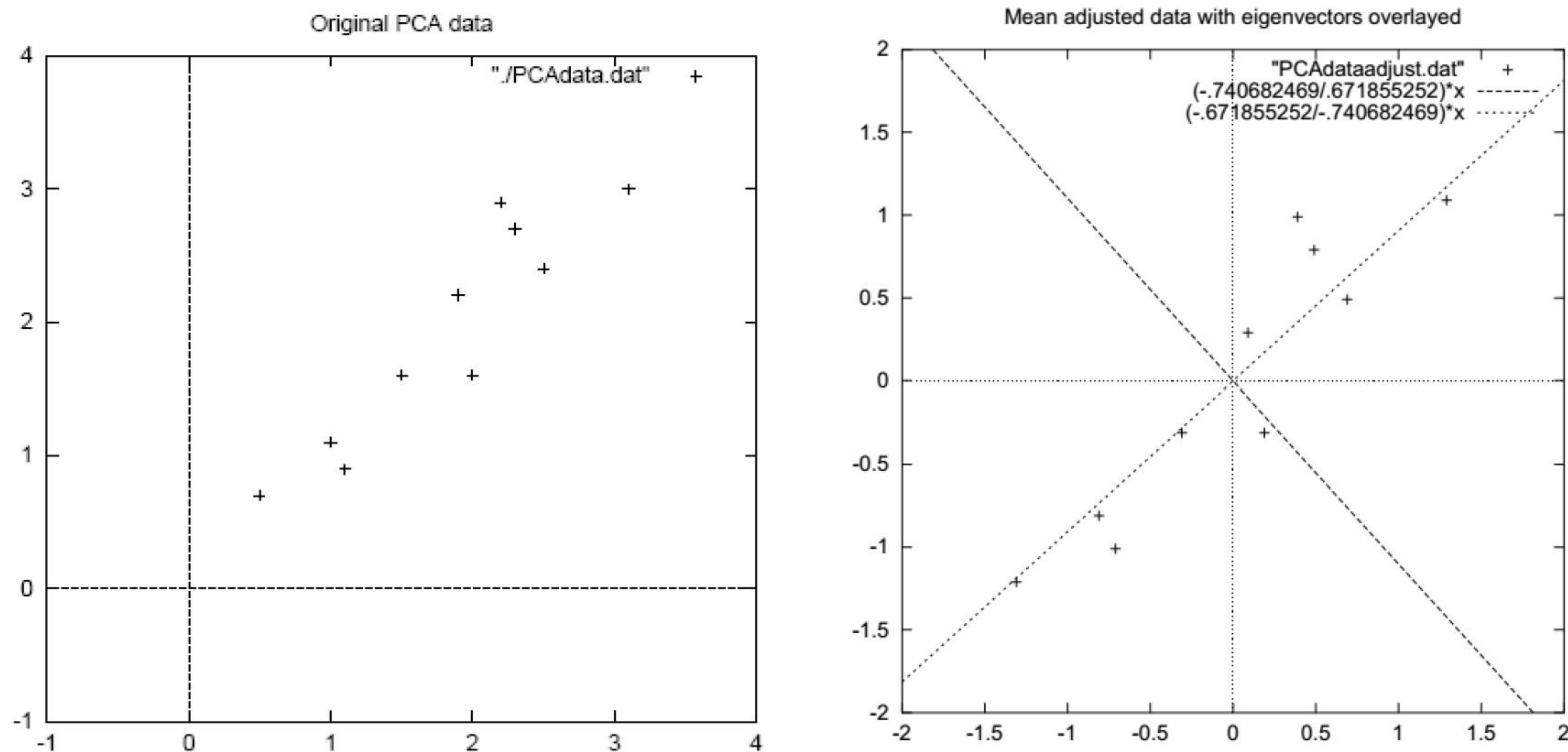


Figure 3.1: PCA example data, original data on the left, data with the means subtracted on the right, and a plot of the data

# Principal Component Analysis algorithm

## 1. Subtract Mean

|        |  | $x$ | $y$ |              |  | $x$   | $y$   |
|--------|--|-----|-----|--------------|--|-------|-------|
| Data = |  | 2.5 | 2.4 | DataAdjust = |  | .69   | .49   |
|        |  | 0.5 | 0.7 |              |  | -1.31 | -1.21 |
|        |  | 2.2 | 2.9 |              |  | .39   | .99   |
|        |  | 1.9 | 2.2 |              |  | .09   | .29   |
|        |  | 3.1 | 3.0 |              |  | 1.29  | 1.09  |
|        |  | 2.3 | 2.7 |              |  | .49   | .79   |
|        |  | 2   | 1.6 |              |  | .19   | -.31  |
|        |  | 1   | 1.1 |              |  | -.81  | -.81  |
|        |  | 1.5 | 1.6 |              |  | -.31  | -.31  |
|        |  | 1.1 | 0.9 |              |  | -.71  | -1.01 |

Subtracting the mean makes variance and covariance calculation easier by simplifying their equations. The variance and co-variance values are not affected by the mean value.

# Principal Component Analysis algorithm

## 2. Calculate the covariance matrix

- Compute “covariance matrix”:

$$\Sigma = \frac{1}{n} \sum_{i=1}^n (x^{(i)})(x^{(i)})^T \quad cov = \begin{pmatrix} .616555556 & .615444444 \\ .615444444 & .716555556 \end{pmatrix}$$

- Since the non-diagonal elements in this covariance matrix are positive, we should expect that both the x and y variable increase together.

- Gives information about shape of data distribution.

$$var(X) = \frac{\sum_{i=1}^n (X_i - \bar{X})(X_i - \bar{X})}{(n - 1)}$$

$$cov(X, Y) = \frac{\sum_{i=1}^n (X_i - \bar{X})(Y_i - \bar{Y})}{(n - 1)}$$

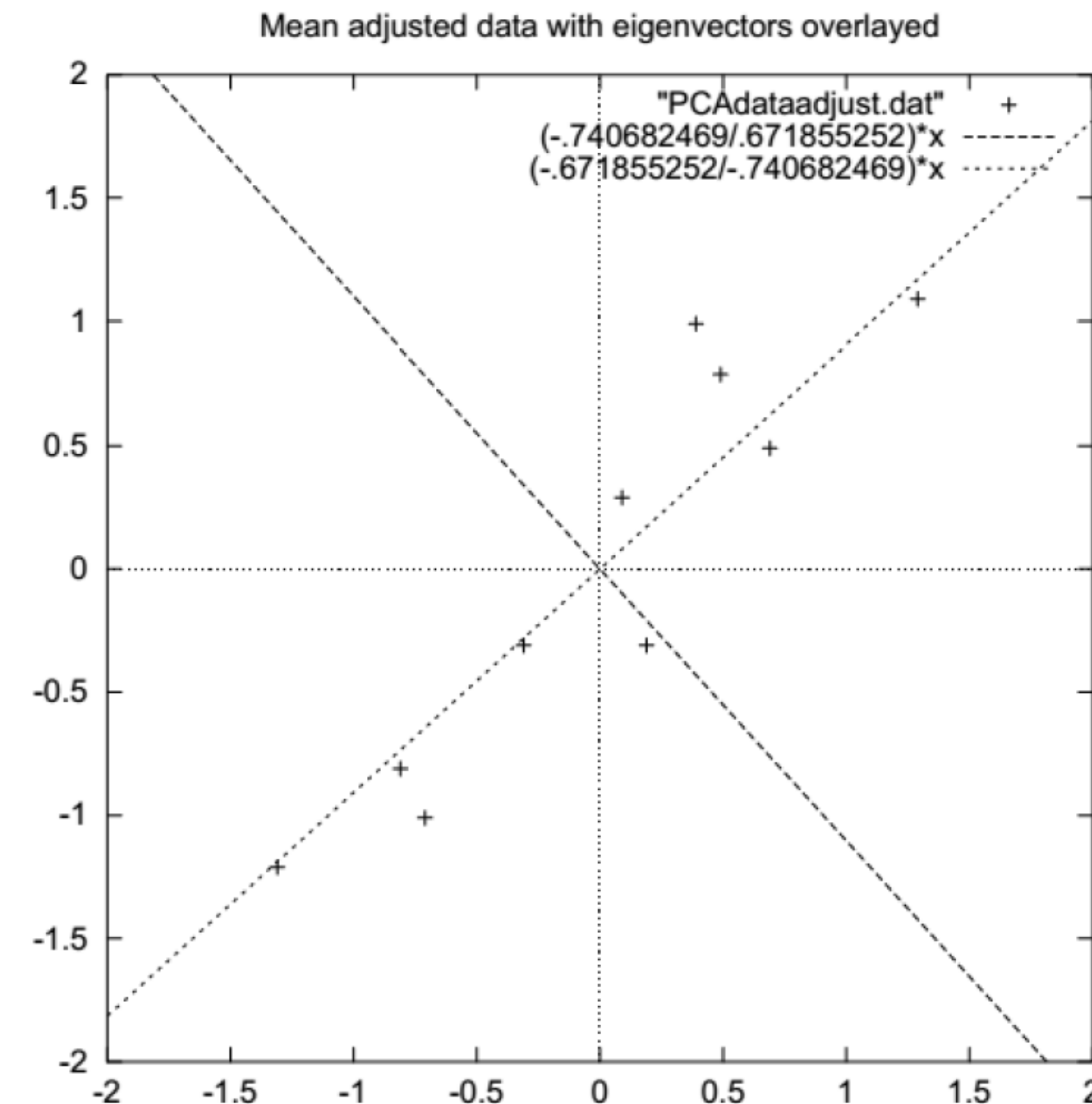
# Principal Component Analysis algorithm

## 3. Calculate the eigenvectors and eigenvalues

- Singular Value Decomposition to get the eigenvectors and eigenvalues:

$$[U, S, V] = \text{svd}(\text{Sigma}) ;$$

$$U = \begin{bmatrix} | & | & \dots & | \\ u^{(1)} & u^{(2)} & \dots & u^{(n)} \\ | & | & \dots & | \end{bmatrix} \in \mathbb{R}^{n \times n}$$



# Principal Component Analysis algorithm

## 4. Choosing components and forming a feature vector

- The eigenvector with the *highest* eigenvalue is the *principle component* of the data set.
- Once eigenvectors are found from the covariance matrix, the next step is to order them by eigenvalue, highest to lowest.
  - This gives you the components in order of significance.
- Form a new feature vector by projecting onto the selected eigenvectors.

# Application of PCA

Visualization *reduce to 2 or 3 features*

Less frequently used for:

- Data compression  
(to reduce storage or transmission costs) *50 → 10*
- Speeding up training of a supervised learning model

$$n = 1000 \rightarrow 100$$

# t-SNE

- t-SNE (t-distributed Stochastic Neighbor Embedding) works in a similar way to PCA but has some key differences:
- Firstly, this is a stochastic method. So, if you run multiple t-SNE plots on the same dataset it can look different.
- Another difference is that this is an iterative method. It works by repeatedly moving datapoints closer or further away from each other depending on how 'similar' they are.
- The new representation is non-linear. This makes it harder to interpret but it can be very effective at 'unravelling' highly non-linear data.
- The main downside to t-SNE is that it is very slow compared to the other techniques.

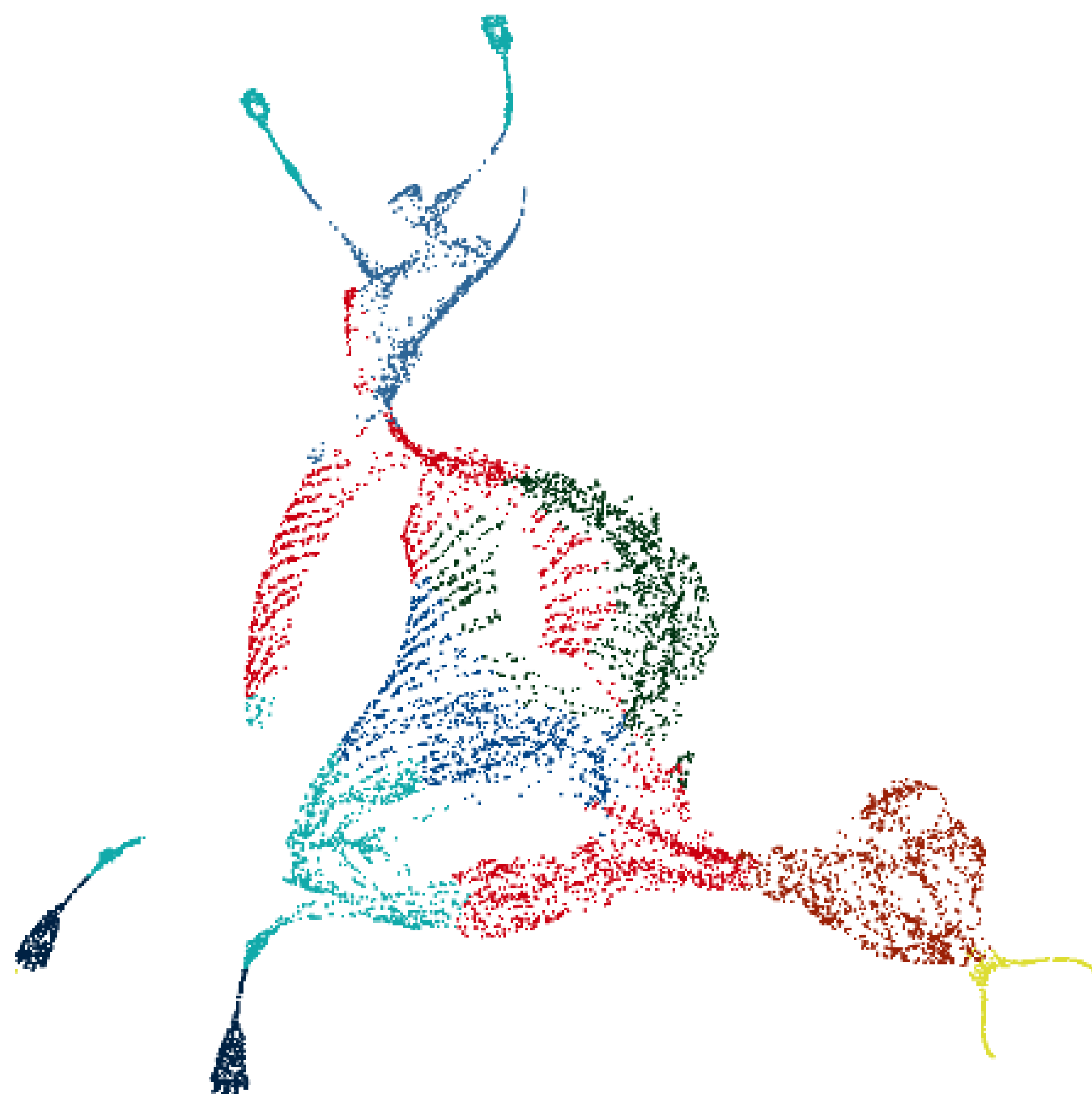
# UMAP

---

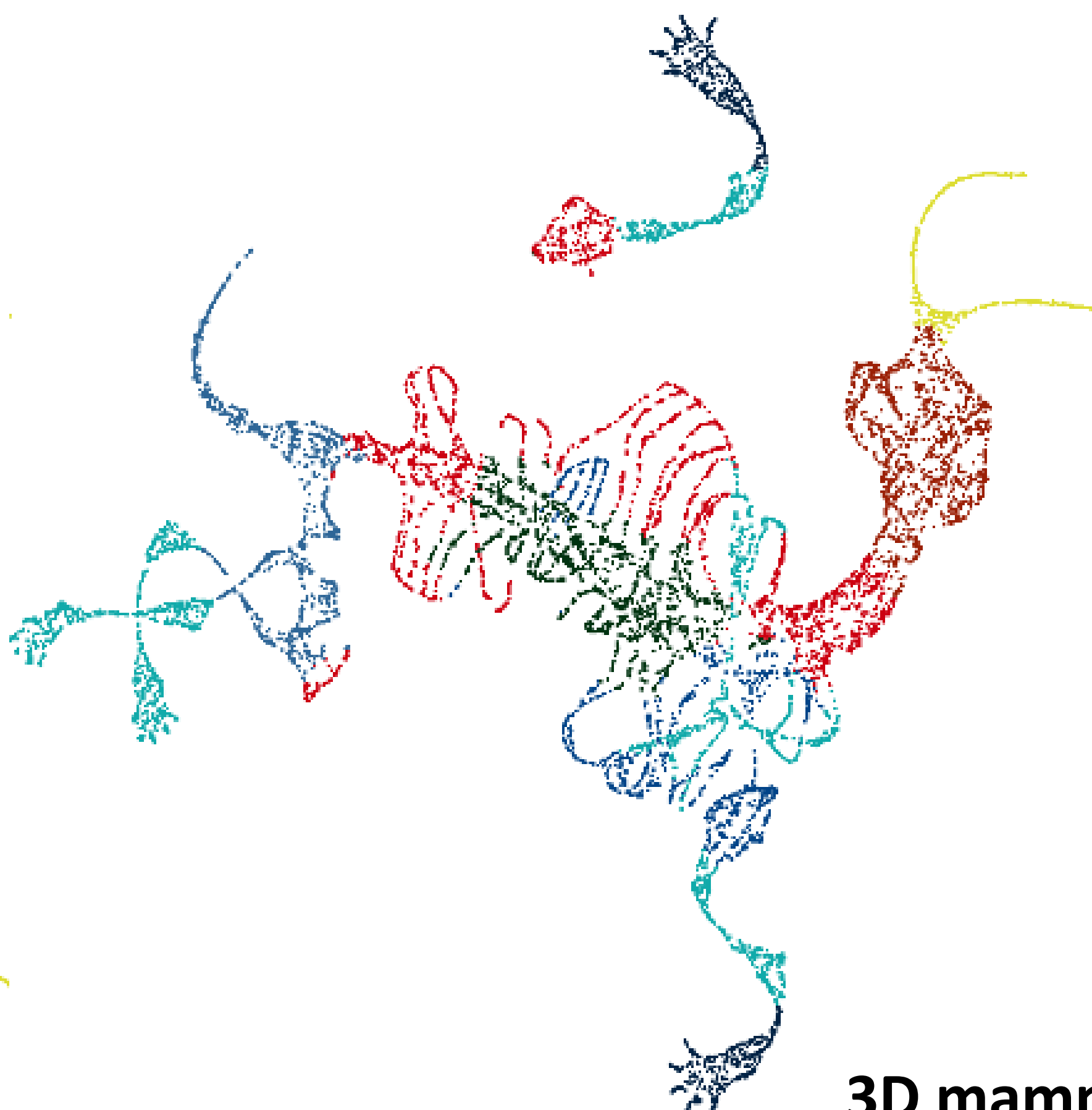
- UMAP, stands for Uniform Manifold Approximation
- It is similar to t-SNE in that it learns a non-linear mapping that preserves clusters, but its main advantage is that it is significantly faster.
- It also tends to do better at preserving global structure of the data compared to t-SNE.

# Example

2D t-SNE projection



2D UMAP projection



**3D mammoth skeleton projected into 2D**

tSNE:      Perplexity 2000      2h 5min

UMAP:      Nneigh 200, mindist 0.25,      3min

# **Comparing PCA, t-SNE and UMAP**

## ➤ **Performance and Scalability**

PCA is computationally efficient and scales well with large datasets.

t-SNE, while powerful for visualization, can be computationally intensive and slow on large datasets.

UMAP strikes a balance, offering speed and scalability while preserving more structure than PCA.

# **Comparing PCA, t-SNE and UMAP**

## ➤ Interpretability

PCA results are easier to interpret because they are linear transformations.

t-SNE and UMAP, being non-linear, produce results that are more complex to interpret but can reveal more intricate structures.

# Comparing PCA, t-SNE and UMAP

## ➤ Application Scenarios

PCA: Use when you need a quick, linear reduction, especially for feature extraction or noise reduction.

t-SNE: Use for detailed visualizations of high-dimensional data, especially when local structure is important.

UMAP: Use for a balance of speed and structural preservation, particularly on larger datasets.



# Thank You

Any questions?

kushol@iut-dhaka.edu

Source & Special Thanks to Andrew Ng (Coursera)  
Machine Learning Course 3

<https://www.youtube.com/@Deeplearningai>